Fundamentos del Lenguaje de Programación Python

Estructuras de selección

Las estructuras de selección permiten la toma de decisiones dentro del flujo de un programa. En Python, se usa la sentencia `if` y sus subordinadas `elif` y `else`.

```
numero = int(input("Ingrese un número: "))

if numero > 0:
    print("El número es positivo")
elif numero < 0:
    print("El número es negativo")
else:
    print("El número es cero")
```

Estructuras de repetición

Las estructuras de repetición permiten ejecutar bloques de código varias veces. En Python se utilizan `for` y `while`.

while

El bucle `while` repite un bloque de código mientras que se cumpla la condición especificada.

```
suma = 0
numero = int(input("Ingrese un número (0 para salir): "))

while numero != 0:
    suma += numero
    numero = int(input("Ingrese un número (0 para salir): "))

print("La suma total es:", suma)
```

range

La función `range` crea una secuencia de números, empezando en 0 por defecto, e incrementándose en uno (1) por defecto, y para antes del número especificado.

```
range(start, stop, step)
```

- `start` es un parámetro opcional de tipo numérico que especifica el número desde que empezar a contar. Por defecto es 0.
- `stop` es un parámetro obligatorio de tipo numérico que especifica el número en el que parar sin incluir este.
- `step` es un parámetro opcional de tipo numérico que especifica el tamaño del paso (incrementación o reducción). Por defecto es 1.

for

El bucle `for` repite un bloque de código mientras que haya elementos que recorrer en una colección.

> *Es importante destacar que en Python, los bucles `for` se comportan de una manera más similar al `foreach` de Java, ya que su objetivo principal es recorrer colecciones. No obstante, gracias a la función `range` se puede hacer que el comportamiento sea algo más similar al de los bucles `for` del resto de lenguajes.*

```
for i in range(1, 6):
    print(i)
```

```
x = range(2, 10, 2)
for n in x:
    print(n)
```

Sentencias de salto

Las sentencias de salto permite alterar el flujo normal de un bucle. En python contamos con:

- `break` que interrumple el bucle.
- `continue` que salta a la siguiente iteración del bucle.
- `pass` instrucción vacía (relleno).

```
for n in range(0, 100, 2):
    if n % 7 == 0:
        print("Primer número divisible por 7 contando de 2 en 2 es:", n)
        break
```

Control de excepciones

El control de excepciones permite manejar errores que surgen durante la ejecución. Se utiliza la estructura `try-except`.

La sentencia `try` va a ejecutar un bloque de código hasta que se lance alguna excepción en ese momento cotejara si está contemplada por alguno de los `except` que se han especificado y en caso de que no lanzará la excepción al programa origen provocando que se paré la ejecución si no es controlada. La sentencia `finally` se ejecutará siempre una vez se haya completado el `try` o algún `except`.

```
try:
    a = int(input("Ingrese el numerador: "))
    b = int(input("Ingrese el denominador: "))
    resultado = a / b
    print("Resultado:", resultado)
except ZeroDivisionError:
    print("Error: No se puede dividir entre cero")
except ValueError:
    print("Error: Debe ingresar un número válido")
finally:
    print("Ejecución finalizada")
```

`raise` una excepción

`raise` es una sentencia que va a lanzar una excepción hacia el programa origen.

> *Se puede elegir no controlar la excepción sino lanzarla hacia arriba (levantarla, `raise`).*

```
x = -1

if x < 0:
  raise Exception("Error: solo números mayores que 0")
```